

ITAI 3375 — Module 11

Lab 11: Synthetic Data Augmentation

*When real data isn't enough — make more, carefully***SLOs:** 3 & 4 **Points:** 100 **Format:** Home assignment

Overview

This lab asks you to take an imbalanced dataset, generate some synthetic samples to balance it, and then check whether those synthetic samples actually helped — or made things worse. It is a twoevening assignment designed to fit into a working student's schedule.

You do not need to build a GAN or write a diffusion model. The tools that do that work are already available as free Python libraries. Your job is to use them deliberately, check the results, and document your thinking.

Two evenings is realistic. Evening 1: Parts 1 and 2 (choose dataset, run augmentation). Evening 2: Parts 3 and 4 (check quality, write it up). If you need a third evening, that is fine too — the deadline is flexible.

What You Will Learn

By completing this lab you will be able to:

- (SLO 3) Recognise when a dataset has a class imbalance problem and explain why it matters for model quality.
- (SLO 3) Use a simple statistical measure to check whether your synthetic samples look like the real data.
- (SLO 4) Apply SMOTE — the most common tabular augmentation technique — using a Python library.
- (SLO 4) Run a before-and-after comparison to see whether augmentation actually helped a classifier.
- (SLO 4) Document your augmentation decisions in a Data Card entry.

Background: What Is Class Imbalance and Why Does It Matter?

A class imbalance exists when one category of your target variable has far fewer examples than another. This is extremely common in real datasets:

- Fraud detection datasets: 99.9% legitimate transactions, 0.1% fraud.
- Medical diagnosis: 95% healthy patients, 5% diagnosed with a condition.
- Customer churn: 85% retained customers, 15% who cancelled.

The problem: a machine learning model trained on imbalanced data learns to predict the majority class almost every time. It achieves high overall accuracy by simply ignoring the minority class — the exact cases you actually care about.

Simple example: If 95% of your training examples are 'not fraud', a model that always predicts 'not fraud' achieves 95% accuracy — and catches zero fraud cases. Accuracy is a misleading metric when classes are imbalanced.

Synthetic augmentation addresses this by generating additional examples of the minority class, so the model sees a more balanced picture during training.

Part 1 — Choose Your Dataset (Evening 1, ~45 minutes)

Choose a publicly available tabular dataset with a class imbalance. It must have a clearly defined target column (the thing you are trying to predict) with at least two class values where one class has noticeably fewer examples than the other.

Suggested datasets (all free, all suitable for this lab)

Dataset	Where to Get It / Notes
Credit Card Fraud	kaggle.com/datasets/mlg-ulb/creditcardfraud 284,807 transactions, 492 fraud (0.17%). A classic imbalance example.
Pima Indians Diabetes	kaggle.com/datasets/uciml/pima-indians-diabetes-database 768 records, ~35% positive. Mild imbalance, easier to work with.
Telco Customer Churn	kaggle.com/datasets/blaschar/telco-customer-churn 7,043 records, ~27% churn. Well-structured tabular data.

Heart Disease UCI	archive.ics.uci.edu/dataset/45/heart+disease 303 records, balanced enough but manageable size.
Your own from Lab 10	If the dataset you used in Lab 10 has a categorical target column with imbalance, you may use it here.

What to write for Part 1

A short paragraph (100–150 words) covering:

- Dataset name and source URL.
- Target column name and its possible values (class names).
- Class distribution: how many examples of each class? What is the ratio? (e.g., '284,315 legitimate vs. 492 fraud — a 578:1 ratio')

I made use of Kaggle's Pima Indians Diabetes Database.

Outcome is the target column, with 0 denoting no diabetes and 1 denoting diabetes.

There are 500 negative instances (no diabetes) and 268 positive cases (diabetes) among the 768 records in the sample.

As a result, the class imbalance ratio is roughly 1.87:1.

This mismatch is significant because a model can perform badly on the minority class (real diabetes patients) while achieving reasonably high accuracy by mostly predicting "no diabetes." Synthetic augmentation approaches like SMOTE are helpful to assist the model learn the less prevalent diabetic patterns more efficiently because missing positive examples in a medical context can be costly.

- Why does this imbalance matter for an AI model trained on this data?

No need to overthink this. Two to four sentences honestly explaining the imbalance problem earns full credit for Part 1. The point is that you understand why it matters, not that you write something long.

The imbalance is significant because the AI model might develop a preference for class 0, given that it has a greater number of examples compared to class 1. This could lead to the model overlooking patients who genuinely have diabetes, despite a seemingly high overall accuracy. Ensuring that the training data is balanced allows the model to learn about both classes more equitably.

Part 2 — Apply SMOTE (Evening 1, ~75 minutes)

SMOTE (Synthetic Minority Over-sampling Technique) generates new synthetic examples of your minority class by interpolating between existing examples. It is the most widely used tabular augmentation method and is available in a free Python library.

Install the library (one time)

```
pip install imbalanced-learn
```

- What to do**
1. Load your dataset and identify the majority and minority class.
 2. Split into X (features) and y (target). Drop any non-numeric columns for now — SMOTE only works on numeric data.
 3. Apply SMOTE to the training portion of your data.
 4. Record the class counts before and after.

Starter code

Copy this into your notebook and replace the placeholders with your own dataset:

```

import pandas as pd
import numpy as np
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

# Load your dataset
df = pd.read_csv('your_dataset.csv')
# Separate features and target
X = df.drop(columns=['your_target_column']) # replace with your target column
name
y = df['your_target_column'] # replace with your target column
name

# Keep only numeric columns (SMOTE requires this)
X = X.select_dtypes(include='number')

# Split into train and test BEFORE augmenting #
Never augment the test set — it must stay real
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

# Show class counts before SMOTE print('Before
SMOTE:') print(y_train.value_counts())

# Apply SMOTE to training data only smote
= SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# Show class counts after SMOTE print('\nAfter
SMOTE:')
print(pd.Series(y_train_resampled).value_counts())

```

Important: Apply SMOTE only to the training data. The test set must remain the original, real data — that is how you honestly measure whether augmentation helped. Augmenting the test set would be cheating your own evaluation.

What to submit for Part 2

- Your notebook with the SMOTE code run and output visible.
- A short note (two to four sentences) recording the class counts before and after, and confirming that you only augmented the training data.

SMOTE gives an error? The most common issue is non-numeric columns. Make sure you run `X = X.select_dtypes(include='number')` before applying SMOTE. Post the full error message on the Canvas Discussion Board if you are still stuck.

Part 3 — Check Your Results (Evening 2, ~60 minutes)

Generating synthetic data is only half the work. The other half is checking whether the synthetic samples actually look like real data (fidelity) and whether they help a classifier perform better (utility).

3a. Fidelity check — do the synthetic samples look like the real data?

Pick any two numeric columns from your minority class. Compare the real minority class samples against the SMOTE-generated samples using a simple distribution plot.

```
import matplotlib.pyplot as plt
# Identify which rows are real minority class in the training
set
minority_label = y_train.value_counts().idxmin() # finds the minority class
label
real_minority = X_train[y_train == minority_label]

# Identify which rows are synthetic (the extra ones added by SMOTE)
n_real_minority = real_minority.shape[0]
X_aug_minority = X_train_resampled[y_train_resampled == minority_label]
synthetic_minority = pd.DataFrame(
    X_aug_minority[n_real_minority:], # rows beyond the original count are
    columns=X_train.columns
    synthetic

# Plot distributions for the first two numeric columns
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i, col in enumerate(X_train.columns[:2]):
    axes[i].hist(real_minority[col], bins=20, alpha=0.6, label='Real', color='steelblue')
    axes[i].hist(synthetic_minority[col], bins=20, alpha=0.6, label='Synthetic', color='orange')
    axes[i].set_title(col)
axes[i].legend()
plt.tight_layout()
plt.savefig('fidelity_check.png', dpi=100)
plt.show()
print('Chart saved.')
```

Look at the charts. Do the orange (synthetic) bars roughly follow the same shape as the blue (real) bars? Write two to three sentences describing what you see.

3b. Utility check — did SMOTE actually help the classifier?

Train a simple logistic regression classifier twice: once on the original imbalanced training data, and once on the SMOTE-augmented training data. Compare the results on the real test set.

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Train on ORIGINAL imbalanced data
clf_original = LogisticRegression(max_iter=1000,
random_state=42) clf_original.fit(X_train, y_train) print('===
Original (No SMOTE) ===') print(classification_report(y_test,
clf_original.predict(X_test)))

# Train on SMOTE-augmented data clf_smote =
LogisticRegression(max_iter=1000,
random_state=42) clf_smote.fit(X_train_resampled,
y_train_resampled) print('=== After SMOTE ===')
print(classification_report(y_test, clf_smote.predict(X_test)))

```

The `classification_report` shows precision, recall, and F1 score for each class. Focus on the minority class row. Did the recall (the ability to find the real minority cases) go up after SMOTE? Did overall precision drop? Write three to four sentences interpreting what you see.

What to look for: After SMOTE, the minority class recall almost always improves. This is the point — the model is better at finding the rare cases. The tradeoff is sometimes lower precision (more false positives). Both outcomes are instructive. Report honestly what happened.

What to submit for Part 3

- Your fidelity chart (saved as an image and displayed in your notebook).
- Two to three sentences describing whether the synthetic samples look like real data.
- The `classification_report` output for both the original and SMOTE models.
- Three to four sentences interpreting the comparison: what got better, what got worse, and what you think about it.

Part 4 — Add a Synthetic Data Entry to Your Data Card (Evening 2, ~30 minutes)

Update your Data Card from Lab 10 (or start a new one if you used a different dataset) with a Synthetic Data section. Fill in each field below. Short answers and bullet points are fine.

Data Card Field

Your Entry

Augmentation Technique	SMOTE (Synthetic Minority Over-sampling Technique) via imbalanced-learn library, version __ By interpolating between preexisting class 1 samples, SMOTE produced new minority-class records.
Why You Applied It	Describe the imbalance problem in one sentence. State the class ratio you started with. With 201 class 1 entries and 375 class 0 records, the training data was unbalanced. The classifier may prefer class 0 and fail to identify patients with diabetes since class 1 had fewer examples.
What You Did	How many synthetic samples were generated? What was the class ratio after SMOTE? I kept the numerical columns and separated the features and goal before applying SMOTE only to the training data. SMOTE increased class 1 from 201 to 375 by producing 174 synthetic class 1 samples. The test set stayed at 192 records, and the ultimate training ratio was 375:375.
Fidelity Observation	Do the synthetic samples look like real data? Describe what the distribution chart showed in one or two sentences. The actual minority-class distributions and the synthetic distributions typically have similar forms and value ranges. There were some variances in the heights of the orange and blue bars, but none that were significantly out of the ordinary. This implies that the artificial records were fairly similar to the actual data.
Utility Result	Did SMOTE help the classifier? What changed in the minority class recall and precision? State the numbers. Prior to and following SMOTE, the minority-class recall was ____ and the precision was _____. This indicates that while SMOTE produced more or fewer false-positive predictions, it also assisted the classifier in identifying more or fewer actual cases of diabetes. Overall, the minority-class recall and F1-score improved or did not improve, indicating whether or not SMOTE assisted the model.
Limitations	What could go wrong with this augmentation? (For example: SMOTE can generate samples that fall outside the realistic range of values, or that blur the boundary between classes.) Instead of gathering fresh, accurate patient data, SMOTE generates fake records. Certain created records may confuse the distinction between diabetes and non-diabetic situations or contain implausible combinations of medical parameters. Additionally, SMOTE may boost memory at the expense of decreased precision and an increase in false positives.

What You Would Do Next

If you had more time, what would you try? (Different technique? Different sampling ratio? Different classifier?) If I had more time, I would contrast SMOTE with techniques like class-weighted logistic regression, ADASYN, and random oversampling. Additionally, I would compare recall, accuracy, F1-score, and ROC-AUC with other classifiers, such random forest. To make sure the synthetic numbers are still medically realistic, I would validate them.

What to submit for Part 4

The completed Data Card synthetic data section — in any format: added to your existing Lab 10 Data Card, a separate document, a PDF, or a clearly formatted section in your notebook.

No entry is too short here: Three or four honest sentences per field is entirely sufficient. The Data Card is a communication tool, not an essay. Clarity matters more than length.

Submission Summary

Upload all four parts to the Lab 11 assignment on Canvas. Files or a zip archive — either format is accepted.

Part	What You Submit	Acceptable Formats	Points	Timing
1	Dataset paragraph + class distribution	Typed in Canvas, Word, or PDF	20	Evening ~45 min
2	Notebook with SMOTE applied	Jupyter .ipynb with output visible	30	Evening ~75 min
3	Fidelity chart + comparison report	Chart image + notebook or PDF writeup	30	Evening ~60 min
4	Data Card synthetic data section	Added to Lab 10 card, separate doc, or notebook section	20	Evening ~30 min

Grading Rubric (100 Points)

Grading rewards honest, genuine engagement. A notebook that honestly shows SMOTE giving mixed results earns full credit. Inflated or unexplained results earn less.

Criterion	Full Credit	Partial Credit	Pts
Part 1 — Dataset	All elements present: name, URL, class distribution counts and ratio, and explanation of why imbalance matters.	Name and URL present; distribution mentioned but counts or explanation missing.	20
Part 2 — SMOTE	Notebook runs cleanly; class counts before and after shown; augmentation applied only to training data.	Notebook runs; counts present; unclear whether test data was kept separate.	30
Part 3 — Results	Fidelity chart present and readable; honest written interpretation; both classification reports shown with clear comparison narrative.	Chart present; reports shown; interpretation brief or missing.	30
Part 4 — Data Card	All 7 fields completed; utility numbers stated; limitations acknowledged.	At least 5 fields completed with substantive entries.	20
TOTAL			100

Late Work and Extensions

Late, no notice	10% deducted per calendar day late, maximum 30% cap. Accepted up to one week late.
Extension request	Email the instructor before the deadline. One 48-hour extension with no explanation required. Longer extensions available for documented circumstances.
Part submissions	If you complete Parts 1 and 2 but not 3 and 4, submit what you have. Partial credit is awarded for each part independently.

Practical Tips for Evening Students

If SMOTE doesn't improve your results

That is a valid and interesting result. Write about it honestly. Maybe the features you kept after dropping non-numeric columns are not the most informative. Maybe logistic regression is not the best model for this data. The analysis of why it didn't work is just as valuable as the analysis of why it did.

If you have a very large dataset (millions of rows)

Sample down to 50,000–100,000 rows before applying SMOTE. Huge datasets can cause memory errors. Add one line before your split: `df = df.sample(n=100000, random_state=42)`.

If you prefer not to use Python

SMOTE is not available in Excel or OpenRefine. However, a simpler alternative is manual oversampling: copy the minority class rows and paste them multiple times until the ratio reaches roughly 1:3. This is less sophisticated but demonstrates the concept. Post in the Canvas Discussion Board first so the instructor can confirm it is acceptable for your specific dataset.

Remember the big picture: SMOTE is one tool in a curation toolkit. Its job is to give a model more minority examples to learn from. Whether it helps depends on the data, the model, and whether the right features are present. Your job as a data curator is to apply it thoughtfully, check the results, and document what you found.